RapidDOC: Riccati-Preconditioned Implicit Differentiation for Differentiable Optimal Control under Operator Mismatch

Xiangda Zhu, Yuankun Chen, Zifei Nie*, and Yi Chang*

***Abstract*— Differentiable optimal control embeds constrained optimization problems in data-driven pipelines by differentiating their optimality conditions and propagating task-loss gradients to model or objective parameters. A key observation is that forward optimization and backward sensitivity computation involve related stage-structured linear systems, making reuse of forward factorization information natural. However, direct reuse is generally invalid, because differences in linearization, barrier terms, regularization, and curvature can create a mismatch between the retained forward operator and the target operator defining the derivative. We therefore propose RapidDOC, a factorization-reuse method that leaves the target adjoint equation unchanged. The retained cache produces a candidate adjoint and, when its target residual exceeds the prescribed tolerance, serves as a fixed right preconditioner for GMRES. A cache-action error operator characterizes directional exactness, the preconditioned iteration, and the relation between target residual and outer-gradient error. An ablation on three constrained robotic OCPs shows that the proposed method achieves lower backward time than both target-system refactorization and a no-reuse iterative method, demonstrating the computational benefit of reusing the retained forward factorizations. Across open- and closed-loop parameter-learning experiments, the proposed method achieves learning performance comparable to the baselines, while yielding the lowest open-loop backward time on all three tasks and reducing total closed-loop solver-call time by 40.5–83.1% relative to the fastest competing method. Finally, deploying the learned controller on a physical vehicle reduces the median figure-eight tracking RMSE from 22.75 to 6.09 cm, demonstrating its potential for hardware deployment.**

***Index Terms*— Differentiable optimization, factorization reuse, implicit differentiation, mobile robotics, imitation learning.**

I. Introduction

Robotic systems often adapt model parameters, objective weights, or reference trajectories as tasks and operating conditions change. Differentiable optimal control enables such adaptation by embedding a constrained OCP in a learning pipeline [1]–[3]. Gradients propagated through the optimized trajectory support system identification, inverse optimal control, learning from demonstrations, and data-driven model predictive control [3], [4]. Their cost, however, includes both the nominal computation performed by the forward solver and a backward sensitivity calculation.

The backward pass creates a specific tension. A structure-exploiting solver has already computed expensive stage-structured Newton or KKT factorization information, so reusing it is numerically attractive. Yet the operator represented by that information need not be the one that defines the desired derivative. Stale linearizations, barrier updates, regularization, and curvature approximations can all separate the cached forward operator from the derivative-defining target operator. Treating the cache as an exact target factorization can then satisfy a different adjoint equation. The issue is therefore not factor reuse alone, but reuse without redefining the derivative.

Sparse nonlinear-programming and structure-exploiting solvers such as IPOPT, acados, HPIPM, and FATROP accelerate nominal optimization using sparse factorizations, condensing, or Riccati recursions [5]–[9]. Differentiable optimization methods compute sensitivities using iterative implicit differentiation, auxiliary optimal control systems, or specialized sensitivity recursions [3], [4], [10]–[12]. Direct factorization reuse is possible when the forward and backward systems are identical or deliberately aligned, as in OptNet and differentiable MPC [1], [2]. sIPOPT reuses IPOPT factorizations for parametric sensitivity analysis [13], while DiffMPC shares an approximate KKT representation across forward and backward Krylov solvers [14]. Conversely, dNMPC constructs the prescribed backward system separately [15]. These methods either align the forward and backward operators for direct reuse or construct the prescribed backward system independently.

We address the intermediate regime in which the retained forward factorization remains useful despite a mismatch with the derivative-defining operator. To address this gap, we propose **RapidDOC**, short for **R**icc**A**ti-**P**reconditioned **I**mplicit **D**ifferentiation for **D**ifferentiable **O**ptimal **C**ontrol under operator mismatch. In the **forward pass**, the OCP solver retains its last available generalized Riccati factorization. In the **backward pass**, **RapidDOC** uses these factors to obtain an adjoint candidate and, when needed, to right-precondition GMRES for the prescribed target adjoint system. The target equation remains unchanged, and its residual determines whether the computed adjoint is accepted.

Figure 1 illustrates this forward–backward interface. The main contributions are:

- We develop a factorization-reuse backward scheme for forward–backward operator mismatch. The prescribed target operator remains the sole definition of the derivative, while the retained generalized Riccati cache provides an adjoint candidate and a fixed right preconditioner. This permits directional direct reuse when valid and cache-guided correction otherwise, without requiring a fresh target-system factorization.
- We introduce a cache-action error operator that provides a unified analysis of direct reuse and cache-guided

*Corresponding authors (e-mail: zifei_nie@jlu.edu.cn).
Supplementary video: https://youtu.be/b53gcrjdS7c.

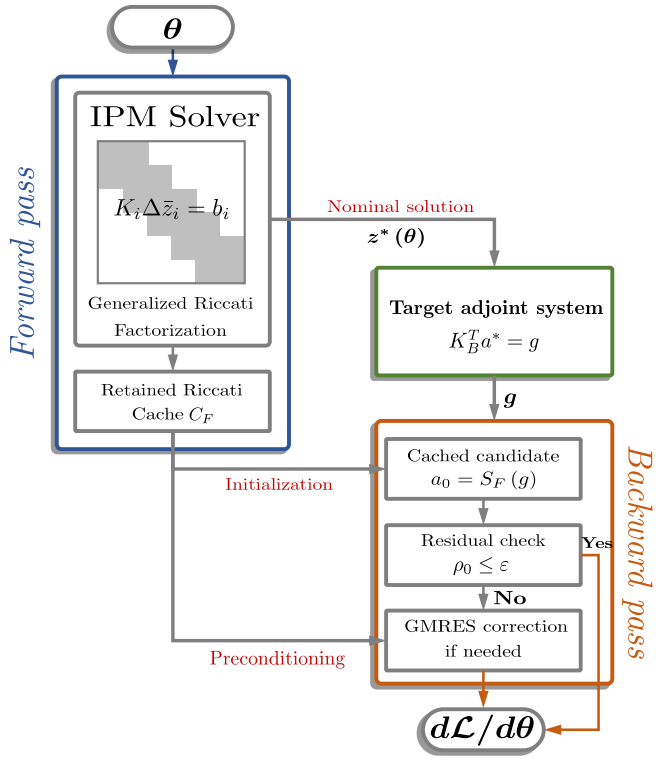


Fig. 1. Proposed factorization-reuse framework. The retained Riccati cache provides an adjoint candidate and right preconditioner, while the target residual determines acceptance.

correction. It characterizes when reuse is exact for a given loss direction, governs the right-preconditioned correction, and connects the certified target residual to outer-gradient error. This analysis shows that reuse depends on the mismatch relevant to the current loss direction rather than global equality of the forward and backward operators.

- We validate the method through a three-way factor-reuse ablation, open- and closed-loop parameter learning on three constrained robotic OCPs, and physical-vehicle deployment. The ablation compares reuse with target-system refactorization and a zero-start, unpreconditioned no-reuse baseline; the learning experiments demonstrate consistent learning behavior with lower backward and repeated-MPC computation time; and physical figure-eight tracking demonstrates hardware deployment potential.

II. Problem Formulation

A. Parameterized Optimal Control

We consider a parameterized finite-horizon nonlinear OCP. At stage k, let $x_k \in \mathbb{R}^{n_x}$ and $u_k \in \mathbb{R}^{n_u}$ denote the state and control input. The parameter vector $\theta \in \mathbb{R}^{n_\theta}$ may enter the dynamics, initial condition, objective, or constraints. For a horizon of N stages, define $\xi := \mathrm{col}(x_0, u_0, \ldots, x_{N-1}, u_{N-1}, x_N)$. For a given θ, the OCP is, with stage-indexed constraints imposed for $k = 0, \ldots, N-1$,

$$\begin{aligned} \min_{\xi} \quad & J(\xi,\theta) := \sum_{k=0}^{N-1} \ell_k(x_k,u_k,\theta) + \ell_N(x_N,\theta) \\ \text{s.t.} \quad & x_0 = \bar{x}(\theta), \\ & x_{k+1} = f_k(x_k,u_k,\theta), \\ & c_k(x_k,u_k,\theta) = 0, \quad h_k(x_k,u_k,\theta) \le 0, \\ & c_N(x_N,\theta) = 0, \quad h_N(x_N,\theta) \le 0. \end{aligned} \tag{1}$$

Here, ℓ_k and ℓ_N are the stage and terminal costs, f_k denotes the discrete-time dynamics, and $\bar{x}(\theta)$ specifies the initial condition. The functions c_k and c_N represent equality constraints, whereas h_k and h_N represent componentwise inequality constraints. These functions are assumed to be twice continuously differentiable on the relevant domain.

To obtain a smooth target for differentiation, we use a fixed-barrier reformulation of the OCP. For a barrier coefficient $\tau > 0$, held fixed during differentiation, define

$$\begin{aligned} \widetilde{J}(\xi,\theta) := J(\xi,\theta) - \tau \sum_{k=0}^{N-1} \mathbf{1}^\top \log\big(-h_k(x_k,u_k,\theta)\big) \\ - \tau \mathbf{1}^\top \log\big(-h_N(x_N,\theta)\big). \end{aligned} \tag{2}$$

The logarithm is applied componentwise on the strictly feasible domain. For fixed τ, let $\xi^\star(\theta)$ denote a local solution of the resulting equality-constrained barrier problem. All derivatives below refer to this finite-τ solution map rather than to the active-set sensitivity of the original inequality-constrained OCP.

B. Adjoint System

Let $q(\xi,\theta)$ collect the initial-state, dynamics, and equality constraint residuals in (1), and let λ denote their multipliers. With $z := \mathrm{col}(\xi,\lambda)$, the first-order optimality residual of the fixed-barrier problem is

$$\mathcal{F}(z,\theta) := \begin{bmatrix} \nabla_\xi \widetilde{J}(\xi,\theta) + \left(\dfrac{\partial q(\xi,\theta)}{\partial \xi}\right)^\top \lambda \\ q(\xi,\theta) \end{bmatrix}. \tag{3}$$

Let $\lambda^\star(\theta)$ denote the multipliers associated with $\xi^\star(\theta)$. The selected primal–dual solution $z^\star(\theta) := \mathrm{col}(\xi^\star(\theta), \lambda^\star(\theta))$ satisfies $\mathcal{F}(z^\star(\theta),\theta) = 0$. We assume that $\mathcal{F}$ is continuously differentiable near this root and that its Jacobian with respect to z is nonsingular. Throughout, the nominal primal–dual solution is required to satisfy these prescribed optimality conditions to the stated forward tolerance. The mismatch studied below concerns the retained linear operator, not a change in the differentiation point.

At the selected solution, define the prescribed backward operators

$$\begin{aligned} K_B &:= \frac{\partial \mathcal{F}}{\partial z} = \begin{bmatrix} H & A^\top \\ A & 0 \end{bmatrix}, \\ G_B &:= \frac{\partial \mathcal{F}}{\partial \theta} = \begin{bmatrix} B \\ C \end{bmatrix}. \end{aligned} \tag{4}$$

Here, H is the Hessian of the fixed-barrier Lagrangian, $A = \partial q/\partial \xi$, and B and C contain the corresponding

derivatives with respect to θ. All quantities are evaluated at $(\xi^\star(\theta), \lambda^\star(\theta), \theta)$ with τ held fixed. The subscript B distinguishes this derivative-defining target operator from the cached forward operator retained by the solver.

The implicit function theorem [16] applied to $\mathcal{F}(z^\star(\theta), \theta) = 0$ gives

$$K_B \frac{\partial z^\star}{\partial \theta} = -G_B. \tag{5}$$

Let $\mathcal{L}(\xi, \theta)$ be a continuously differentiable outer loss. Because $\mathcal{L}$ does not depend on the equality multipliers, define the loss-induced adjoint right-hand side

$$g := \mathrm{col}\big(\nabla_\xi \mathcal{L}, 0\big). \tag{6}$$

The prescribed target adjoint is the solution of

$$K_B^\top a^\star = g. \tag{7}$$

The resulting outer gradient is

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\theta} = \nabla_\theta \mathcal{L} - G_B^\top a^\star. \tag{8}$$

The backward task is therefore to compute the adjoint $a^\star$ defined by (7) efficiently while preserving K_B as the derivative-defining operator.

III. Methodology

The method separates the solver-owned cache from the derivative-defining operator. The forward pass retains matrix-dependent generalized Riccati information; the backward pass exposes it only as an approximate inverse action and accepts an adjoint only under the prescribed target residual.

A. Forward Pass

The forward pass computes the nominal solution by applying a primal–dual interior-point method to the smooth fixed-barrier KKT conditions defined in Section II. Under the standard regularity conditions of interior-point methods, solutions of the barrier subproblems approach a KKT point of the original inequality-constrained problem as τ decreases [5]. For finite τ, however, the barrier problem defines a smooth approximation. Throughout this work, both the nominal solution and the prescribed backward derivative refer to this same fixed-τ solution map.

At iteration i, the forward solver computes the step $\Delta\bar{z}_i$ from the stage-structured reduced Newton system

$$K_i \Delta\bar{z}_i = b_i, \tag{9}$$

where K_i is the reduced Newton operator obtained from the current smooth KKT conditions, $\Delta\bar{z}_i$ contains the solver's internal primal–dual increments, and b_i is the iteration-dependent right-hand side. The matrix-dependent part of this system is factorized through a generalized Riccati recursion.

The generalized Riccati recursion separates matrix-dependent eliminations from the vector recursions induced by b_i [9]. Its reusable state comprises stagewise Riccati factors, equality-constraint eliminations, coordinate transformations, and primal–dual recovery maps, rather than a single canonical matrix factorization. Right-hand-side-dependent vector recursions are therefore recomputed for each new right-hand side.

Let i_F denote the iteration of the last successful forward factorization. At termination, we retain its matrix-dependent cache,

$$\mathcal{C}_F := \mathcal{C}_{i_F}. \tag{10}$$

The converged nominal solution $z^\star$ satisfies the prescribed fixed-barrier optimality map $\mathcal{F}(z^\star, \theta) = 0$ to the stated forward tolerance. Accordingly, the backward target K_B is the Jacobian of the same smooth optimality map evaluated at $z^\star$.

Nevertheless, the retained cache $\mathcal{C}_F$ need not represent K_B. It may originate from an earlier iterate, a different barrier parameter, or a system containing regularization, curvature approximations, and iterate-dependent inequality eliminations. Therefore, the forward solution and backward derivative are aligned through the same fixed-barrier optimality map, while the retained linear-algebraic cache may still be mismatched with the prescribed backward operator. The backward pass exploits this cache only as approximate inverse information and never uses it to redefine the derivative.

B. Backward Pass

The backward pass must reuse the retained cache without allowing it to define the derivative. To enforce this separation, we expose the cache through the fixed inverse action

$$\mathcal{S}_F(v) := \mathrm{CachedSolve}(\mathcal{C}_F, v). \tag{11}$$

The retained successful factorization is assumed to define a fixed nonsingular linear action on the primal–dual space. Concretely, `CachedSolve` maps the supplied target-coordinate right-hand side through the cached stagewise transformations and eliminations, performs the fixed vector recursions, and applies the cached recovery maps to return a primal–dual vector in the ordering of z. No explicit forward or target KKT matrix is formed, and all matrix-dependent cached quantities remain fixed throughout the backward computation. Although the canonical KKT operator in (4) is symmetric, we retain the transpose in the adjoint equations to make their reverse-mode origin explicit; it does not require a distinct transpose factorization.

With this fixed action in place, a nonzero loss seed g first produces the cached adjoint candidate

$$a_0 := \mathcal{S}_F(g). \tag{12}$$

Rather than treating this candidate as the target adjoint, we evaluate it using the prescribed target operator:

$$r_0 := g - K_B^\top a_0, \qquad \rho_0 := \frac{\|r_0\|_2}{\|g\|_2}. \tag{13}$$

It is accepted only when $\rho_0 \le \varepsilon$, where $\varepsilon > 0$ is the target-residual tolerance.

Before constructing a correction for an rejected candidate, we first characterize the mismatch responsible for its target residual. Define the cache-action error operator

$$E_F := I - K_B^\top \mathcal{S}_F. \tag{14}$$

Algorithm 1 Backward Pass

Input: Target action $v \mapsto K_B^\top v$, G_B, g, $\nabla_\theta \mathcal{L}$, cache action $\mathcal{S}_F$, and tolerance ε
Output: Gradient estimate $\widehat{d}_\theta$

if $\|g\|_2 = 0$ **then**
  return $\nabla_\theta \mathcal{L}$
end if
$a_0 \leftarrow \mathcal{S}_F(g)$
$r_0 \leftarrow g - K_B^\top a_0$, $\rho_0 \leftarrow \|r_0\|_2 / \|g\|_2$
if $\rho_0 \le \varepsilon$ **then**
  $\widehat{a} \leftarrow a_0$
else
  Initialize GMRES for $K_B^\top \mathcal{S}_F(y) = r_0$ with $y_0 = 0$
  $j \leftarrow 0$
  while $\rho_j > \varepsilon$ **do**
    $j \leftarrow j + 1$ and advance GMRES to y_j
    $a_j \leftarrow a_0 + \mathcal{S}_F(y_j)$
    $r_j \leftarrow g - K_B^\top a_j$, $\rho_j \leftarrow \|r_j\|_2 / \|g\|_2$
  end while
  $\widehat{a} \leftarrow a_j$
end if
$\widehat{d}_\theta \leftarrow \nabla_\theta \mathcal{L} - G_B^\top \widehat{a}$
return $\widehat{d}_\theta$

This definition uses only algorithm-available actions and does not introduce an implicit matrix representation of the cache.

Proposition 1 (Cache-action mismatch characterization): Assume that K_B is nonsingular and $\mathcal{S}_F$ is a fixed nonsingular linear action. The cached candidate satisfies

$$r_0 = E_F g. \tag{15}$$

Consequently,

$$a_0 = a^\star \quad \Longleftrightarrow \quad E_F g = 0. \tag{16}$$

Moreover, the right-preconditioned target operator has the form

$$K_B^\top \mathcal{S}_F = I - E_F. \tag{17}$$

In exact arithmetic, let r_j denote the target residual after j iterations of unrestarted GMRES applied from a zero correction. Then

$$\|r_j\|_2 \le \|E_F^j r_0\|_2 \le \|E_F\|_2^{j+1} \|g\|_2. \tag{18}$$

Proof: By definition of E_F and $a_0 = \mathcal{S}_F(g)$,

$$r_0 = g - K_B^\top \mathcal{S}_F(g) = E_F g.$$

Nonsingularity of K_B implies that $r_0 = 0$ if and only if a_0 is the unique solution of $K_B^\top a = g$. Furthermore,

$$K_B^\top \mathcal{S}_F = I - E_F.$$

GMRES minimizes its residual over polynomials $p \in \Pi_j$ satisfying $p(0) = 1$. Selecting $p_j(t) = (1-t)^j$ gives $p_j(I - E_F) = E_F^j$, and hence

$$\|r_j\|_2 \le \|E_F^j r_0\|_2.$$

The second inequality follows from $r_0 = E_F g$. ∎

Proposition 1 links the two uses of the cache through the same error operator. First, $E_F g$ determines whether direct reuse is exact for the current loss direction. Second, E_F determines the form of the right-preconditioned target operator and provides one admissible GMRES residual bound. This is a structural characterization rather than a general GMRES convergence theorem: $\|E_F\|_2 < 1$ is sufficient, but not necessary, for effective correction. In particular, exact direct reuse is seed-dependent; a mismatched cache can still return the exact adjoint for a specific loss direction when $E_F g = 0$.

We now use this characterization to correct a candidate that fails the target-residual test. When $\rho_0 > \varepsilon$, write the corrected adjoint as $a = a_0 + \delta a$. Substitution into the target equation gives

$$K_B^\top \delta a = r_0. \tag{19}$$

We parameterize the correction using the cached inverse action,

$$\delta a = \mathcal{S}_F(y), \tag{20}$$

which yields

$$K_B^\top \mathcal{S}_F(y) = r_0, \qquad a = a_0 + \mathcal{S}_F(y). \tag{21}$$

Because $\mathcal{S}_F$ is nonsingular, this change of variables is bijective and therefore preserves the original correction equation. Each Krylov product evaluates $v \mapsto K_B^\top \mathcal{S}_F(v)$: the target supplies the matrix action and the cache supplies fixed right preconditioning. Although K_B is symmetric in the canonical coordinates of (4), the right-preconditioned operator $K_B^\top \mathcal{S}_F$ is generally nonsymmetric. We therefore use GMRES [17] rather than a symmetry-dependent Krylov method. In the implementation, the target action is evaluated directly from packed stage Hessian and dynamics-Jacobian blocks. Thus, "matrix-free" refers to the global target KKT matrix: neither that matrix nor its factorization is formed.

Although GMRES operates in the preconditioned variable y, every candidate is accepted according to its residual under the original target operator:

$$r_j := g - K_B^\top a_j, \qquad \rho_j := \frac{\|r_j\|_2}{\|g\|_2}. \tag{22}$$

The returned adjoint $\widehat{a}$ must satisfy $\rho_j \le \varepsilon$.

The certified adjoint gives the outer-gradient estimate $\widehat{d}_\theta = \nabla_\theta \mathcal{L} - G_B^\top \widehat{a}$. To relate the residual certificate to the gradient error, define $r := g - K_B^\top \widehat{a}$ and $d_\theta^\star := \nabla_\theta \mathcal{L} - G_B^\top a^\star$. Since $r = K_B^\top (a^\star - \widehat{a})$,

$$\widehat{d}_\theta - d_\theta^\star = G_B^\top K_B^{-\top} r. \tag{23}$$

Consequently,

$$\|\widehat{d}_\theta - d_\theta^\star\|_2 \le \varepsilon \|G_B^\top K_B^{-\top}\|_2 \|g\|_2. \tag{24}$$

Thus, the target residual certifies satisfaction of the prescribed adjoint equation, while its implication for the outer gradient also depends on the conditioning of K_B and the coupling through G_B.

Algorithm 1 summarizes candidate construction, target-residual testing, and cache-guided correction.

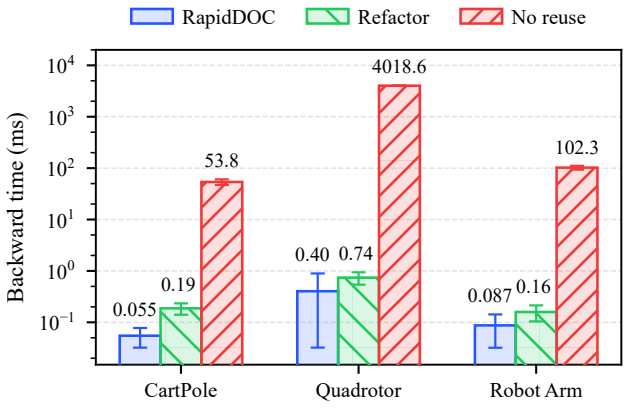


Fig. 2. Backward time in the factorization-reuse ablation. Error bars show one standard deviation over 32 instances; the vertical axis is logarithmic.

Candidate construction requires one cache application and one target product. With m correction iterations, the backward pass uses $1+m$ cache applications and at most $1+2m$ target products, in addition to the Arnoldi updates. It avoids refactorizing K_B, although the benefit remains mismatch- and conditioning-dependent.

IV. Experiments

The experiments evaluate the computational benefit of factorization reuse, its effectiveness in open- and closed-loop parameter learning, and the transfer of the learned controller to a physical vehicle. The forward solver uses a tolerance of 10^{-8}, and every backward computation reaches a target relative-residual tolerance of 10^{-6}. We report backward time for the factor-reuse ablation and forward and backward solver times for the learning experiments. All computational experiments run single-threaded on an Intel Core i5-14600KF under Windows 11.

A. Factorization-Reuse Ablation

This ablation evaluates whether the structured information retained by the forward pass reduces backward computation relative to reconstructing the target factorization or discarding the cache entirely.

We compare three backward strategies. RapidDOC uses the retained cache for candidate construction and, when required, fixed right preconditioning. Refactor discards the cache and constructs a new generalized Riccati factorization for the target system. No reuse applies zero-start, unpreconditioned GMRES to the target system. All variants use the same forward result, target operator, adjoint right-hand side, and stopping criterion.

We evaluate 32 instances for each of CartPole, Quadrotor, and Robot Arm. Their horizons are 30, 50, and 35, respectively, with corresponding target-adjoint dimensions of 278, 1424, and 358. The reported times cover only the backward computation, including target factorization for Refactor and cache application and correction for RapidDOC.

As shown in Fig. 2, RapidDOC achieves the lowest backward time on all three tasks. Its advantage over Refactor shows that the retained forward structure avoids reconstructing and factorizing the target system. Its advantage over No reuse further shows that the cache provides useful inverse information for the target backward computation. The ablation therefore confirms the computational value of retaining the forward factorization under a common target operator and stopping criterion.

B. OCP Parameter Learning from Demonstrations

We next evaluate factorization reuse in complete OCP parameter-learning pipelines. For the jth initial condition ζ_j, let $\xi_j^\star(\theta) := \xi^\star(\theta;\zeta_j)$ denote the trajectory generated by the learner and ξ_j^{ref} the corresponding demonstration. The objective parameters are learned by minimizing

$$\mathcal{L}_{\mathrm{demo}}(\theta) = \frac{1}{2N_{\mathrm{demo}}} \sum_{j=1}^{N_{\mathrm{demo}}} \left\| \xi_j^\star(\theta) - \xi_j^{\mathrm{ref}} \right\|_W^2 , \tag{25}$$

where $\|e\|_W^2 = e^\top W e$ and W accounts for component scales and horizon normalization.

Teacher and learner share the dynamics and constraints but use different objective parameters. We consider CartPole, Quadrotor, and Robot Arm tasks with five, six, and five log-parameterized objective weights, respectively. All methods use the same demonstrations, initialization, loss, and Adam updates [18]. We compare RapidDOC with SafePDP [4], a CasADi–IPOPT reverse-mode pipeline [19], and acados [6].

1) Open-Loop Learning: The open-loop experiment uses 32 training and 16 validation demonstrations. Each OCP has $N = 40$ and a sampling interval of 0.05 s. All methods start from the same seed-2 initialization and are trained for 800 iterations with a learning rate of 2×10^{-3}.

Figure 3 shows closely aligned learning trajectories across methods, indicating that they identify comparable objectives from the demonstrations. RapidDOC also achieves the lowest backward time on all three tasks. By reusing the structured factorizations produced by the current forward computation, the backward pass avoids constructing a separate factorization and reduces the work required for correction.

The comparatively high acados time on Quadrotor is dominated by its forward computation, consistent with the higher cost of exact-Hessian SQP on this larger and more strongly coupled problem. Overall, the results show that factorization reuse reduces the backward cost while preserving the observed open-loop learning behavior.

2) Closed-Loop Learning: The closed-loop experiment applies the same learning procedure through 50 state-coupled MPC steps. Each OCP uses $N = 40$ and a sampling interval of 0.05 s. All methods are trained from the same initialization for 500 epochs with a learning rate of 5×10^{-3}.

Figure 4 shows that the learned trajectories approach their demonstrations and that the learning curves remain closely aligned across methods. The behavior observed in the open-loop experiment therefore persists when successive OCPs are coupled through closed-loop state propagation.

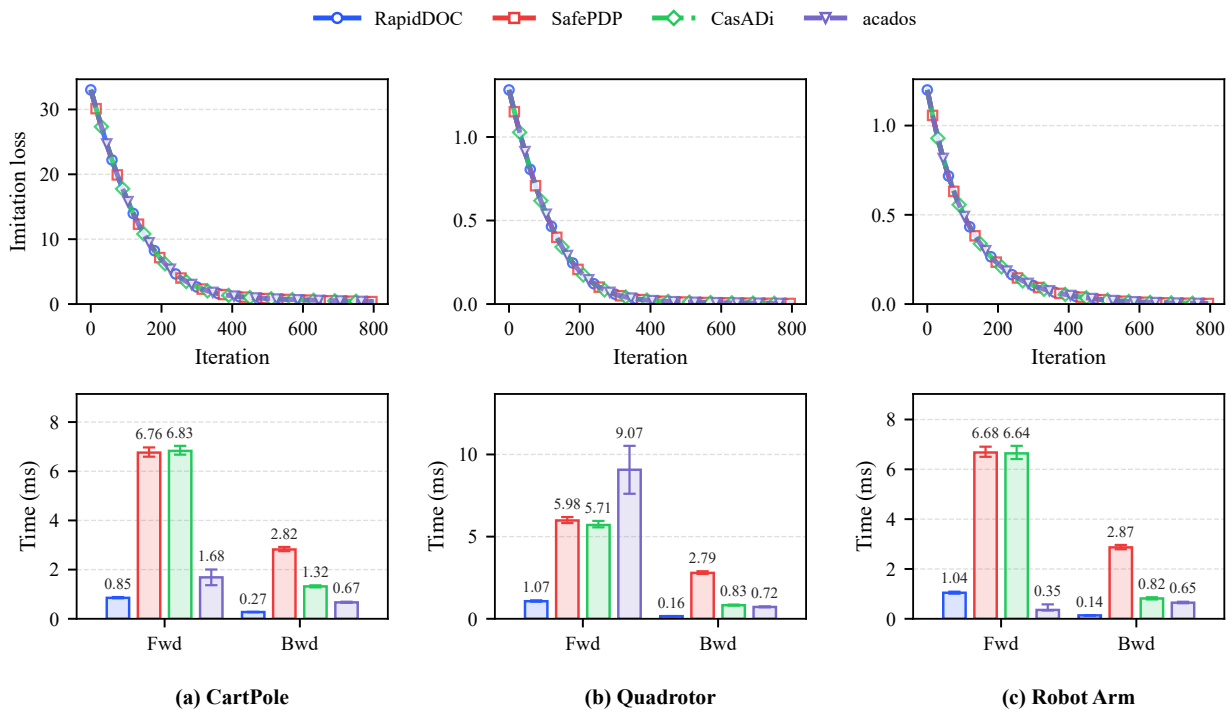


Fig. 3. Open-loop parameter learning. Top: trajectory-matching loss. Bottom: forward and backward solver time; whiskers show interquartile ranges.

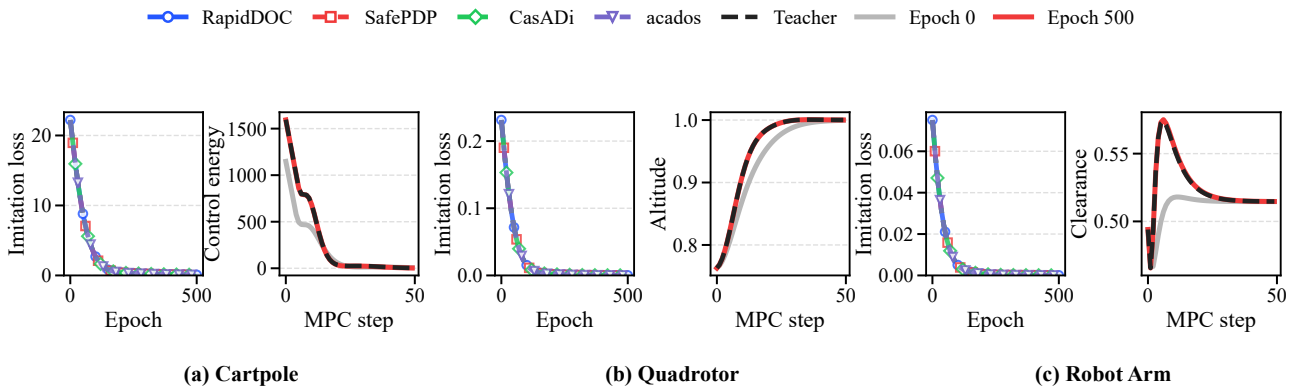


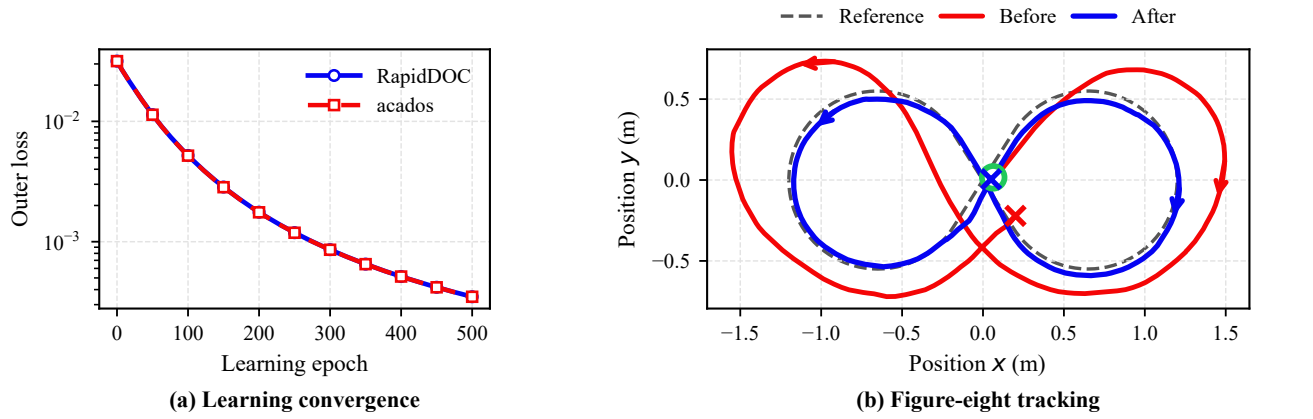
Fig. 4. Closed-loop parameter learning. Left: training loss. Right: teacher and learned trajectories before and after training.

Table I shows that RapidDOC achieves the lowest total solver time on all three tasks. The forward factors are updated and reused at every MPC step, so the computational benefit accumulates over the rollout. These results show that factorization reuse remains effective as the state, trajectory, and optimization problem evolve during closed-loop learning.

C. Physical Vehicle Demonstration

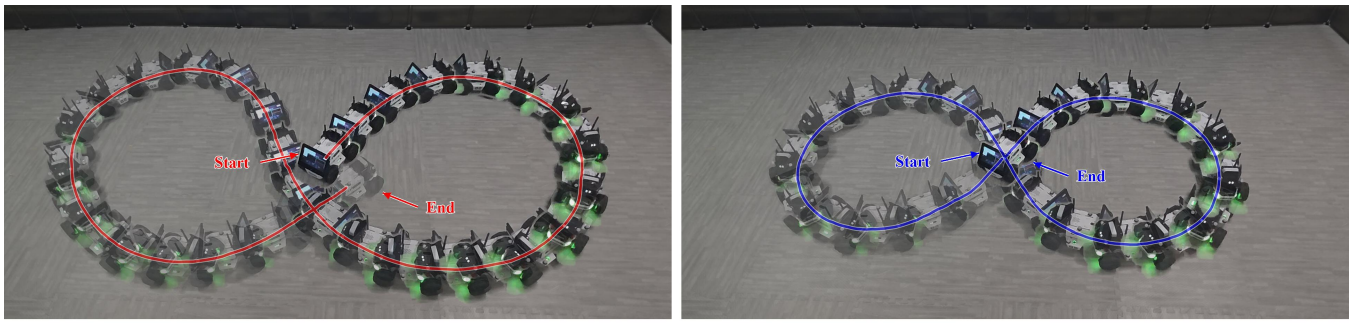
To evaluate the potential of the proposed method for real-world deployment, we consider a figure-eight imitation-learning task in which an initially poor tracking controller is improved using a demonstrated trajectory. The controller employs a five-state kinematic bicycle model comprising planar position, yaw, speed, and reference phase, with acceleration and steering as inputs. Four log-parameterized objective weights penalizing position, heading, speed, and steering are optimized to match the demonstrated trajectory. The MPC horizon is $N = 25$, the sampling interval is 0.1 s, and the fixed barrier coefficient is $\tau = 10^{-3}$. Training is performed in simulation for 500 epochs using 707-step closed-loop rollouts.

Figure 5 reports the learning curves and the tracking behavior obtained before and after training. We compare RapidDOC with acados using the same demonstration and parameter initialization. The two methods produce nearly identical decreases in the imitation loss, indicating that the gradients computed by RapidDOC lead to a learning outcome consistent with the reference solver. The trajectory comparison further shows that the controller obtained from the initial parameters deviates substantially from the demonstrated figure-eight path, whereas the learned controller follows the reference much more closely. Thus, the benefit of the



(a) Learning convergence **(b) Figure-eight tracking**

Fig. 5. Figure-eight imitation learning: (a) training losses of RapidDOC and acados; (b) trajectories before and after learning.



(a) Before learning (b) After learning

Fig. 6. Physical figure-eight tracking before and after learning over the same 70.5 s interval.

TABLE I

MEDIAN FORWARD, BACKWARD, AND TOTAL SOLVER TIME PER MPC STEP DURING CLOSED-LOOP LEARNING (MS).

Task	Method	Fwd.	Bwd.	Total
CartPole	RapidDOC	**0.77**	**0.13**	**0.91**
	SafePDP	6.18	2.78	8.97
	CasADi	6.06	2.51	8.59
	acados	2.60	2.79	5.40
Quadrotor	RapidDOC	**1.07**	**0.79**	**1.86**
	SafePDP	6.02	2.79	8.80
	CasADi	5.58	2.09	7.69
	acados	1.40	3.96	5.34
Robot Arm	RapidDOC	0.92	**0.63**	**1.60**
	SafePDP	5.96	2.83	8.81
	CasADi	5.62	2.14	7.76
	acados	**0.27**	2.42	2.69

TABLE II

MEDIAN SOLVER TIME PER MPC STEP ON THE ONBOARD PLATFORM (MS).

Method	Fwd.	Bwd.	Total
RapidDOC	**1.45**	**1.73**	**3.17**
acados	2.08	2.36	4.44

computed derivatives is reflected not only in the loss value but also in the resulting control behavior.

We next evaluate the computational cost of both methods on the onboard computing platform. Table II reports the median forward, backward, and total solver-call times per MPC step during figure-eight learning. Backward time includes the forward recomputation required by closed-loop reverse accumulation. RapidDOC requires 1.45 ms for the forward pass and 1.73 ms for the backward pass, compared with 2.08 ms and 2.36 ms for acados. Consequently, the median total solver-call time is reduced from 4.44 to 3.17 ms per MPC step. This result is consistent with the computational behavior observed in the preceding experiments and shows that the proposed forward and backward computations can be executed efficiently on the onboard processor.

Finally, the initial and learned controllers are deployed in the onboard ROS system without further parameter updates, with the first MPC input applied at 10 Hz. We conduct three complete runs before learning and two after learning. The sequences in Fig. 6 use the same camera viewpoint and scale, without zooming or rescaling; the corresponding planar trajectories are shown in Fig. 5(b). The learned

controller follows the reference more closely, reducing the median position RMSE from 22.75 cm to 6.09 cm. Given the limited number of runs, this experiment demonstrates hardware deployment potential rather than statistical superiority.

V. Conclusion

We presented **RapidDOC**, a factorization-reuse method for differentiable OCPs under forward–backward operator mismatch. **RapidDOC** uses retained generalized Riccati factors to initialize and precondition the backward solve, while the prescribed target operator remains the sole definition of the derivative. A three-way ablation demonstrates the computational benefit of factorization reuse. Open-loop, closed-loop, and physical-vehicle experiments further show consistent parameter learning and the potential for hardware deployment. This study considers a fixed-barrier formulation and naturally occurring mismatches; future work will address active-set changes and adaptive reuse in embedded online differentiation.

References

[1] B. Amos and J. Z. Kolter, "OptNet: Differentiable optimization as a layer in neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 70, 2017, pp. 136–145.

[2] B. Amos, I. D. J. Rodriguez, J. Sacks, B. Boots, and J. Z. Kolter, "Differentiable MPC for end-to-end planning and control," in *Advances in Neural Information Processing Systems*, vol. 31, 2018, pp. 8299–8310.

[3] W. Jin, Z. Wang, Z. Yang, and S. Mou, "Pontryagin differentiable programming: An end-to-end learning and control framework," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 7979–7992.

[4] W. Jin, S. Mou, and G. J. Pappas, "Safe pontryagin differentiable programming," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 16 034–16 050.

[5] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical Programming*, vol. 106, no. 1, pp. 25–57, 2006.

[6] R. Verschueren *et al.*, "acados—a modular open-source framework for fast embedded optimal control," *Mathematical Programming Computation*, vol. 14, no. 1, pp. 147–183, 2022.

[7] G. Frison and M. Diehl, "HPIPM: A high-performance quadratic programming framework for model predictive control," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 6563–6569, 2020.

[8] L. Vanroye, A. S. Sathya, J. D. Schutter, and W. Decré, "FATROP: A fast constrained optimal control problem solver for robot trajectory optimization and control," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2023, pp. 10 036–10 043.

[9] L. Vanroye, J. D. Schutter, and W. Decré, "A generalization of the riccati recursion for equality-constrained linear quadratic optimal control," *Optimal Control Applications and Methods*, vol. 45, no. 1, pp. 436–454, 2024.

[10] A. Agrawal, S. Barratt, S. Boyd, E. Busseti, and W. M. Moursi, "Differentiating through a cone program," *Journal of Applied and Numerical Optimization*, vol. 1, no. 2, pp. 107–115, 2019.

[11] Y. Chen, Z. Nie, X. Gong, Y. Hu, and H. Chen, "A Gauss–Newton-induced structure-exploiting algorithm for differentiable optimal control," 2025, arXiv:2512.19447.

[12] Y. Chen, Z. Nie, X. Gong, Y. Hu, and H. Chen, "PANDA: A matrix-free differentiable NMPC solver via proximal averaged quasi-newton with adaptive linesearch algorithm," 2026, arXiv:2608.16280.

[13] H. Pirnay, R. López-Negrete, and L. T. Biegler, "Optimal sensitivity based on IPOPT," *Mathematical Programming Computation*, vol. 4, no. 4, pp. 307–331, 2012.

[14] E. Adabag, M. Greiff, J. Subosits, and T. Lew, "Differentiable model predictive control on the GPU," in *Proceedings of the International Conference on Learning Representations*, 2026.

[15] J. Frey *et al.*, "Differentiable nonlinear model predictive control," 2025, arXiv:2505.01353.

[16] A. L. Dontchev and R. T. Rockafellar, *Implicit Functions and Solution Mappings: A View from Variational Analysis*, 2nd ed. New York, NY, USA: Springer, 2014.

[17] Y. Saad and M. H. Schultz, "GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems," *SIAM Journal on Scientific and Statistical Computing*, vol. 7, no. 3, pp. 856–869, 1986.

[18] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proceedings of the International Conference on Learning Representations*, 2015.

[19] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi—a software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.